



Hinweis

Durch die Größe der ROS2-Packages wurden diese nicht angehängt. Die Code-Lösungen liegen in den beigefügten Dateien `FakeEncoder.cpp` und `recorder.cpp`.

Aufgabe A2.1 (Transformationen)

Gegeben:

$$T = \begin{pmatrix} a & b & c \\ d & e & f \\ g & h & i \end{pmatrix}$$

(a) Was ist eine Transformation?

- Mathematische Operation zur Veränderung von Punkten, Vektoren oder Koordinatensystemen im Raum.
- Dient der Darstellung von Translationen (Verschiebungen), Rotationen (Drehungen) und Skalierungen.
- Typischer Anwendungsfall: Umrechnung von Sensordaten in das Koordinatensystem des Roboters.

(b) Rotationskomponenten

- Obere linke 2x2 Untermatrix.
- Einträge: **a, b, d, e** (Hier stehen üblicherweise Sinus- und Kosinus-Werte).

(c) Translationskomponenten

- Rechte Spalte der Matrix (Verschiebung in x- und y-Richtung).
- Einträge: **c, f**.

(d) Transformation eines Punktes im 2D Raum

- Punkt $\begin{pmatrix} x \\ y \end{pmatrix}$ muss in **homogene Koordinaten** umgewandelt werden.
- Eine 1 wird als dritte Dimension angehängt: $\begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$.
- Anschließend Multiplikation mit der Matrix T .

Aufgabe A2.2 (Joint State Publisher - Theorie)

- Um die Gelenke in RViz zu animieren, muss ein `JointState`-Publisher erstellt werden.
 - Ein Timer aktualisiert die Winkel periodisch um 1 Grad ($\frac{\pi}{180}$) und hält sie im Intervall $[0, 2\pi]$.
 - **Code-Lösung:** Siehe beiliegende Datei `FakeEncoder.cpp`.
 - Sie Bild `A2.2.png`
-



Aufgabe A2.3 (Absolutgeber - Theorie)

(a) Wie ist der Roboter gefahren?

- **Linker Motor (M_l):** Zählt aufwärts (101 -> 110 -> 111 -> 000...). Dies entspricht laut Text einer Vorwärtsbewegung. Er dreht sich pro Zeitschritt um exakt 1 Sektor.
- **Rechter Motor (M_r):** Zählt abwärts (000 -> 111 -> 110 -> 101...). Da die Motoren entgegengesetzt montiert sind, bedeutet dieses Abwärtszählen, dass das rechte Rad **ebenfalls vorwärts** fährt! Es ändert den Sektor aber nur jeden zweiten Zeitschritt (halbe Geschwindigkeit).
- **Fazit:** Da beide Räder vorwärts fahren, das linke Rad aber doppelt so schnell ist wie das rechte, fährt der Roboter eine **Linkskurve**.

(b) Zurückgelegte Strecke

- Durchmesser $d = 0.3$ m
- Umfang $U = \pi \cdot 0.3$ m ≈ 0.9425 m
- Strecke pro Sektor ($\$ 45^\circ$): $\$ U / 8 \approx 0.1178$ „m“ $\$$
- Hinweis: Beide Strecken sind positiv, da beide Räder vorwärts rollen.

T	0	1	2	3	4	5	6	7	8	9
D_l (m)	0.000	0.118	0.236	0.353	0.471	0.589	0.707	0.825	0.942	1.060
D_r (m)	0.000	0.000	0.118	0.118	0.236	0.236	0.353	0.353	0.471	0.471

(c) Probleme der Kodierung

- **Problem:** Bei der Standard-Binärkodierung ändern sich beim Sektorübergang oft mehrere Bits gleichzeitig.
- **Konkretes Beispiel:** Beim Übergang von 111 -> 000 (siehe linkes Rad bei $T = 3$ zu $T = 4$) ändern sich alle 3 Bits.
- **Ursache & Folge:** Da physikalische Sensoren nie auf die Millisekunde exakt synchron schalten, entstehen kurzzeitig falsche Zwischenzustände (z.B. liest der Sensor kurz 110 oder 100). Das verfälscht die Positions- und Geschwindigkeitsmessung massiv.
- **Lösung:** Nutzung eines **Gray-Codes**. Bei diesem Code ändert sich pro Sektorübergang immer exakt nur 1 Bit, wodurch solche Zwischenzustände unmöglich werden.

Aufgabe B2.1 (Differenzialantrieb)

Startwerte: $x_0 = 0$, $y_0 = 0$, $\theta_0 = 0^\circ$, $b = 30$ cm Formeln: $\Delta d = \frac{s_l + s_r}{2}$ und $\Delta \theta = \frac{s_r - s_l}{b}$

(a) Posenberechnung

Zeitpunkt $t = 1$: ($s_r = 6$ cm, $s_l = 10$ cm)

- $\Delta d = \frac{10+6}{2} = 8$ cm
- $\Delta \theta = \frac{6-10}{30} = -\frac{4}{30} \approx -0.1333$ rad (ca. -7.64°)
- $x_1 = 0 + 8 \cdot 1 = 8$ cm
- $y_1 = 0 + 8 \cdot 0 = 0$ cm
- $\theta_1 = 0 - 0.1333 = -0.1333$ rad
- **Pose 1:** (8 cm, 0 cm, -0.1333 rad)



Zeitpunkt t = 2: ($s_l = 5$ cm, $s_r = 7$ cm)

- $\Delta d = \frac{5+7}{2} = 6$ cm
- $\Delta\theta = \frac{7-5}{30} = \frac{2}{30} \approx 0.0667$ rad
- $x_2 = 8 + 6 \cdot \cos(-0.1333) \approx 8 + 6 \cdot 0.991 = 13.94$ cm
- $y_2 = 0 + 6 \cdot \sin(-0.1333) \approx 0 + 6 \cdot (-0.132) = -0.79$ cm
- $\theta_2 = -0.1333 + 0.0667 = -0.0666$ rad
- **Pose 2:** (13.94 cm, -0.79 cm, -0.0666 rad)

(b) Fehlerwirkung (Rotation vs. Translation)

- **Rotationsfehler** wirken sich langfristig viel stärker aus.
- Translationsfehler summieren sich nur linear auf.
- Ein Rotationsfehler ändert die globale Fahrtrichtung → der resultierende Positionsfehler (Abbe-Fehler) wächst mit jedem gefahrenen Meter an.

Aufgabe B2.2 (Lokalisierungsqualität)

ROS2-Node zum Aufzeichnen

- Node abonniert das Topic `/odom`.
- Extrahiert die Werte `msg->pose.pose.position.x` und `y`.
- Schreibt die Koordinaten zeilenweise in eine Textdatei.
- **Code-Lösung:** Siehe beiliegende Datei `recorder.cpp`.

Bei welchen Operationen ist der Fehler hoch?

- **Bei Drehungen / Rotationen** wächst der Odometrie-Fehler besonders stark.
- Gründe:
 - Radschlupf (Durchdrehen) auf dem Untergrund.
 - Ungenauigkeiten in der Annahme der exakten Spurweite (b).
 - Bodenunebenheiten (Rad dreht frei in der Luft, was fälschlicherweise als Vorwärtsbewegung integriert wird).